# CODESTREAK

## MS-1: Problem Statement and Design Document

*CS602 C Project 2026 — AI-Enabled Web Application*
Submission milestone: 15 October 2026

## 1. Project Overview

### 1.1 Website Name

CodeStreak — AI-Enabled Competitive Programming Practice Platform

### 1.2 Team Members

| Roll No | Name | Role / Main Responsibility |
|---|---|---|
| **CS26MT015** | Avishkar | Backend, REST API and database |
| **CS26MT017** | Shivam | Frontend and user experience |
| **CS26MT023** | Kiran | AI integration, testing and CI support |

### 1.3 Project Description

Students who practise competitive programming may find it difficult to keep their practice organized, track solved problems, and notice which topics need more work. CodeStreak is planned as a web application where users can browse a curated set of programming problems, record their attempts, set daily goals, and review their practice history.

The application will also include an AI-assisted hint feature. It will give users hints or explanations about a selected problem instead of immediately presenting a complete solution. The first version will focus on practice tracking and learning support; it will not compile or execute submitted code.

### 1.4 Target Users

Primary users are university students and beginner-to-intermediate competitive programmers who want a structured way to practise algorithms, build consistent habits, and identify topics that need more attention. Secondary users are administrators or maintainers who curate problems, topics, and metadata.

### 1.5 Project Goals and Scope

- Provide a searchable, curated problem library with difficulty levels and topic tags.
- Allow registered users to record attempts and mark problems as solved.
- Calculate daily activity, current streak, longest streak, and progress statistics.
- Allow users to set daily goals and maintain a simple practice plan.
- Offer AI-generated, problem-contextual hints and explanations through a server-side API.
- Provide secure authentication, input validation, automated tests, CI, and containerized local startup.
- Out of scope for the initial version: a custom compiler, arbitrary code execution sandbox, hidden-test judging infrastructure, live contests, and a complex social network.

## 2. Dataset Plan

### 2.1 Dataset Source and License

The initial problem dataset will be a team-curated dataset created specifically for CodeStreak. The team will author original problem statements, sample inputs/outputs, constraints, difficulty labels, and topic tags. Synthetic user activity and sample profiles may be generated for development and demonstration using scripts or Faker. No third-party problem statement will be copied without checking its license and attribution requirements.

Source link: Not applicable to the original team-authored problem dataset. If a public dataset is added later, its exact source URL, license, attribution, and any transformations will be recorded here before use. Development-only synthetic records will be generated by the team and will not represent real people.

### 2.2 Dataset Fields

| Entity / Data | Example Fields | Used By |
|---|---|---|
| **Problems** | problem_id, title, statement, constraints, sample_input, sample_output, difficulty, created_at, is_published | Problem Library, Problem Details, AI Assistant |
| **Tags** | tag_id, name, description | Problem filters, topic analytics, recommendations |
| **Problem–Tag mapping** | problem_id, tag_id | Many-to-many relationship between problems and topics |
| **Users** | user_id, username, email, password_hash, created_at, consent_status | Registration, login, profile and account management |
| **User problem status** | user_id, problem_id, status, first_attempt_at, solved_at, notes | Problem status, history, progress statistics |
| **Daily activity** | user_id, activity_date, problems_solved_count | Streak calendar, current/longest streak and dashboard |
| **Practice plans** | plan_id, user_id, plan_date, target_count, completed_count | Daily goals and practice planning |
| **AI interaction metadata** | interaction_id, user_id, problem_id, request_type, created_at, outcome | AI usage limits, troubleshooting and evaluation; avoid storing unnecessary personal data |

### 2.3 Dataset Construction and Maintenance

- The team will create an initial, manageable collection of original problems with a range of topics and difficulty levels.
- Problem statements and sample cases will be reviewed by another team member for clarity and consistency.
- A seed script will populate the database during container startup or initialization so a fresh clone can run without manual data import.
- Synthetic users and activity records, if used, will be clearly identified as demo data.
- The dataset will be stored in the repository in a documented format or loaded through a seed script. Dataset changes will be version-controlled.

## 3. Content Structure and Sitemap

The following sitemap describes the planned navigation. Public visitors can browse published problems; login is required for personal tracking, practice plans, and AI usage.

```
codestreak/
```

```
frontend/                # React + Vite source
backend/
  app/
    main.py              # FastAPI application entry point
    routes/               # API route modules
    models/               # Database models
    schemas/              # Request/response validation
    services/             # Application logic
database/                 # Optional seed data / SQL
Dockerfile               # Multi-stage React + Python build
docker-compose.yml       # App + MySQL services
.env.example
.gitignore
README.md
```

Planned distinct pages: (1) Dashboard, (2) Problem Library, (3) Problem Details, (4) Streak Calendar, (5) Progress Analytics, (6) Practice Plan, (7) AI Practice Assistant, (8) Profile & Settings, (9) Login, and (10) Registration.

## 4. Page-by-Page Functionality

### 1. Dashboard (/dashboard)

Shows current and longest streak, total solved problems, today's goal, recent activity, and a suggested next problem. Users can navigate to practice, progress, and streak pages.

### 2. Problem Library (/problems)

Lists published problems with search and filters for difficulty, topic, and status. Visitors can browse problems; signed-in users can see their own status.

### 3. Problem Details (/problems/{id})

Displays the statement, constraints, examples, difficulty, and tags. Signed-in users can record an attempt, mark a problem solved, add a note, and request a contextual AI hint. The initial version does not run submitted code.

### 4. Streak Calendar (/streaks)

Displays activity by date, current streak, longest streak, and daily completion counts. Users can inspect historical consistency.

### 5. Progress Analytics (/progress)

Summarizes completed problems by topic and difficulty, along with weekly/monthly activity. Users can identify topics that need more practice.

### 6. Practice Plan (/plan)

Lets users set a daily target and create or update a simple date-based practice plan. Users can compare planned and completed counts.

### 7. AI Practice Assistant (/assistant)

Accepts a selected problem and a hint/explanation request. It returns contextual hints or conceptual explanations and shows an appropriate message if the AI service is unavailable.

### 8. Profile & Settings (/profile)

Shows account information and personal summary. Users can edit permitted profile fields, change preferences, log out, and request account/data deletion.

### 9. Login (/login)

Accepts credentials, validates input, and creates an authenticated session or token. Invalid credentials produce a safe error message.

### 10. Registration (/register)

Creates an account after validating fields and collecting consent. Passwords are stored only as salted hashes using a suitable password-hashing algorithm.

## 5. User Stories

1. As a visitor, I want to browse published problems without creating an account so that I can decide whether the platform is useful to me.
2. As a learner, I want to filter problems by topic and difficulty so that I can practise at an appropriate level.
3. As a registered user, I want to record that I attempted or solved a problem so that my progress reflects my work.
4. As a learner, I want to view my daily activity calendar so that I can understand my practice consistency.
5. As a learner, I want to see my current and longest streak so that I can stay motivated to practise regularly.
6. As a user, I want to set a daily problem-solving goal so that I can structure my practice sessions.
7. As a learner, I want to see progress by topic and difficulty so that I can identify areas where I need more practice.
8. As a learner, I want to request a hint for a selected problem so that I can make progress without immediately seeing a complete solution.
9. As a registered user, I want to edit my profile and delete my account so that I can manage my information and privacy.
10. As an administrator, I want to create, update, publish, and categorize problems so that the problem library remains accurate and organized.

## 6. AI Feature Proposal

### 6.1 Feature Name and Purpose

AI Problem-Solving Assistant: The assistant will provide progressive hints and conceptual explanations grounded in the selected problem's statement, constraints, examples, and topic tags. The goal is to help students reason about an approach rather than automatically reveal a complete solution. The first release will focus on text hints; code review and automatic judging are not part of the initial scope.

### 6.2 Planned Model / API

Proposed provider: Open Ai API, accessed only from the Python backend. The exact model will be selected after checking current availability, free/student-tier limits, latency, and terms at implementation time. The provider choice is a proposal and may be changed if a suitable API is unavailable.

### 6.3 Data Sent to the AI Service

- The selected problem's statement, constraints, sample examples, and topic tags.
- The user's hint request and an optional short description of where they are stuck.
- Optionally, the user's own solution attempt only if the user explicitly submits it for feedback; this is not required for the initial feature.
- No passwords, email addresses, authentication tokens, or other direct personal identifiers will be sent to the model.

### 6.4 Request Flow

1. The frontend sends the problem ID and hint request to a protected backend endpoint.
2. The backend validates the request, enforces input length and per-user rate limits, and retrieves the approved problem context.
3. The backend sends a carefully constructed prompt to the AI provider using an API key stored in an environment variable.
4. The backend validates and returns the response to the frontend. The interface labels the response as AI-generated and encourages users to verify explanations.
5. The application does not depend on the AI service for core browsing, tracking, authentication, or streak functionality.

### 6.5 Failure Handling, Guardrails and Evaluation

- Use request timeouts and handle provider errors, rate limits, quota exhaustion, and malformed responses.
- If the AI service fails, display a clear message and allow the rest of the application to continue working.
- Keep API keys out of source control and frontend code; use environment variables locally and GitHub Secrets in CI/deployment.
- Limit request length and rate-limit hint requests per user. Treat problem text and user-provided content as untrusted input and instruct the model not to follow instructions embedded in problem content that conflict with its task.
- Mock model calls in unit and integration tests so CI does not depend on the external provider or consume quota.
- Create an evaluation set of at least 20 representative problem/hint requests. Review whether hints are relevant, correct, appropriately progressive, and avoid immediately exposing full solutions; document failures and limitations honestly.
- Record only the minimum metadata needed for troubleshooting and evaluation. Explain data handling to users and do not send direct personal identifiers to the provider.

## 7. Initial Technical Design

| Layer | Proposed Technology / Responsibility |
|---|---|
| **Frontend** | React with Vite; responsive HTML/CSS/JavaScript UI |
| **Backend** | Python with FastAPI; validation and REST endpoints |
| **Database** | MySQL 8; relational schema and seed data |
| **ORM and migrations** | SQLAlchemy and Alembic |
| **API contract** | OpenAPI 3.x specification, viewable in Swagger UI |

| | |
|---|---|
| **Testing** | pytest and pytest-cov; Playwright for end-to-end tests |
| **CI** | GitHub Actions for linting and automated tests on pushes and pull requests |
| **Packaging** | Dockerfiles and Docker Compose; fresh clone should start with docker compose up |
| **AI** | Server-side Open Ai API integration, subject to provider/model verification |

## 8. GitHub Repository : https://github.com/cs26mt015/CODESTREAK

## 9. Risks and Mitigation

| Risk | Mitigation |
|---|---|
| **Scope grows beyond six weeks** | Prioritize problem browsing, tracking, streaks, authentication, and the required AI feature; defer optional features. |
| **AI gives incorrect or overly revealing hints** | Use problem context, progressive-hint instructions, a 20+ case evaluation set, and honest limitations. |
| **AI provider unavailable or quota exhausted** | Timeouts, friendly fallback messages, mocked tests, and no dependency of core features on AI. |
| **Incorrect streak calculations** | Define one activity day per user-local date, test date boundaries and consecutive-day cases, and derive results from stored activity. |
| **Sensitive data exposure** | Hash passwords, protect endpoints, validate inputs, keep secrets server-side, minimize data sent to AI, and support account deletion. |
| **Unreliable setup on a fresh clone** | Use Docker Compose, migrations, seed scripts, documented environment variables, and CI checks. |